Infosys Springboard Virtual Internship-6.0

DAY-1

- The Day-1 of the Infosys Internship Program is about the Introduction to the “Receipt & Invoice Digitizer”.
- In the Session it is about to Convert a photo(or) pdf of a bill into Structured data.
- It includes Vendor Name, Date, Invoice Number, Line items like Item, quantity, price etc., Total Amount, Tax
- So, it is about the Day-1 of the “Infosys Springboard Virtual Internship”.

DAY-2

- The Day-2 of the Infosys program is about the “Team Dividing and Discussion about the Project”.
- The Mentor has discussed about the project in detailed manner and in understandable way.
- It’s all about the Project’s Aim, Process, Goal.
- It is also about the Real-world problem of the project, Planning, Objective, Step-step by Development, What Ai tools we have to use, Research and Analysis etc.

DAY-3

Git

- A version control system used to track changes in files and source code.

GitHub

- An online platform to host Git repositories and collaborate with others.

GitLab

- A Git-based DevOps platform with built-in CI/CD and project management tools.

Git Commands

git add .

- Adds all modified files in the current directory to the staging area.

git add app.py

- Adds only app.py to the staging area.

git clone https://…

- Creates a local copy of a remote repository from a URL.

git pull

- Fetches changes from the remote repository and merges them into the current branch.

git fetch --all

- Downloads updates from all remote branches
- Does NOT merge changes automatically

git branch

- Shows all local branches

git branch --all

- Shows local + remote branches

git checkout branchname

- Switches to an existing branch

git checkout -b newbranchname

- Creates a new branch and switches to it

git commit -m "message"

- Saves staged changes to the repository with a message.

git push

- Uploads local commits to the remote repository (GitHub/GitLab).

Files

- app.py
- main.py
- app1.py
- main5.py

DAY-4

SDLC Roles (Software Development Life Cycle)

1. Functional Consultant / Business Analyst

- Gathers and analyzes business requirements from stakeholders
- Prepares the Functional Specifications Document (FSD), which explains *what* the system should do

2. Developer / Programmer

- Converts functional requirements into technical solutions
- Prepares the Technical Specifications Document (TSD)
- Performs Unit Testing to verify individual modules

3. Tester / QA Team

- Ensures the application meets quality standards
- Prepares Test Scripts based on requirements
- Executes tests and documents Test Results
- Identifies and reports bugs

4. Administration Team

- Manages servers, databases, access control, and deployments
- Ensures system availability, security, and backups

5. Product / Project Management

- Plans, monitors, and controls the project or product lifecycle
- Manages timelines, cost, scope, and risk

6. Sales and Marketing

- Promotes the product and gathers customer feedback
- Supports business growth and market positioning

7. Human Resources (HR)

- Handles recruitment, onboarding, training, and employee relations
- Manages performance and compliance

8. Documentation Team

- Prepares user manuals, technical documents, and help guides
- Ensures clear and accurate documentation

9. Support Team

- Handles customer issues, bug fixes, and maintenance

Main Servers in SDLC

1. **Development Server**
    - Used by developers for coding and unit testing
2. **Quality (QA) Server**
    - Used by testers for functional, integration, and system testing
3. **Production Server**
    - Live environment used by end users

SDLC Models

1. Waterfall Model

- Linear and sequential approach
- Each phase must be completed before moving to the next

2. Agile Methodologies

- Focuses on flexibility, customer feedback, and quick delivery
- Work is done in short cycles called sprints

Agile Roles

1. Product Owner

- Defines and prioritizes the product backlog
- Represents customer and business needs

2. Scrum Master

- Facilitates Agile processes
- Removes impediments and ensures Scrum practices

3. Development Team

- Cross-functional team that designs, develops, tests, and delivers increments of the product

DAY-5

API

1. An API is a bridge that allows two applications to talk to each other.
2. API stands for Application Programming Interface.

How an API Works:

1. Client sends a request
2. API receives the request
3. Server processes the request
4. Server sends a response
5. Client gets the response

HTTP Methods Used in APIs

1. **GET – (Retrieve Data)**

   Used to fetch data

   Does NOT change data in the server

   Data is sent in URL

2. **POST – (Create New Data)**

   Used to send data to server

   Creates new resource

3. **PUT – (Update Entire Data)**

   Used to update existing resource

   Replaces the full data

4. **PATCH – Update Partial Data**

   Used to update only some fields

   Does not replace full data

Flask API

A Flask API is an API built using Flask, which is a lightweight Python web framework.

Flask is used to create web applications and REST APIs easily.

Flask Imports & Setup

1. Install Flask

- pip install flask

2.Basic Flask Imports

- from flask import Flask, request, jsonify

  Flask → Creates the Flask application

  request → Reads data sent by the client

  jsonify → Sends JSON response

3. Create Flask App

- app = Flask(__name__)

4. Create a Route (API Endpoint)

```
@app.route("/hello", methods=["GET"])
def hello():
    return jsonify({"message": "Hello Flask"})
```

5. Run the Flask App

```
if __name__ == "__main__":
    app.run(debug=True)
```

Complete Minimal Flask API

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route("/", methods=["GET"])
def home():
    return jsonify({"message": "Flask API is running"})

if __name__ == "__main__":
    app.run(debug=True)
```

FastAPI

1.FastAPI is a Python web framework used to build APIs (Application Programming Interfaces).

2.Uses Python type hints for automatic data validation.

3.Provides automatic API documentation.

4.Commonly used for web apps, mobile backends, and ML APIs.

Simple FastAPI Example

```
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
def read_root():
    return {"message": "Hello FastAPI"}
@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}
```

Postman

- Postman is a tool used to test APIs.
- It allows sending HTTP requests like GET, POST, PUT, DELETE.

HTTP Request

An HTTP Request is a message sent by a client (browser or app) to a server asking for data or requesting an action.

WebSocket Request

A WebSocket request is a request used to create a continuous, two-way connection between a client and a server.

ASSIGNMENT — 1
Create a Python program that builds an API using either Flask or FastAPI.

DAY-6

1. Login / Registration Page – Fields

Registration Form

- Name
- Email
- Password
- Preferred Language

Login Form

- Email / Username
- Password
- Login with Google

2. Backend (API Layer)

- Built using FastAPI
- **Handles:**
    - User registration
    - User login
    - Google login
    - File upload
    - Dashboard APIs

Security

- JWT (JSON Web Token) for authentication
- Password Hashing (never store plain passwords)

3. Password Handling

- User enters password
- Password is hashed
- Hashed password is stored in database
- During login → hashed password is verified

4. Database Options

You can use any one:

- SQLite (small projects)
- MySQL
- PostgreSQL
- MongoDB

5. User Table (Example)

Column Name	Description
user_id	Unique user ID
username	User name
email	User email
password	Hashed password

6. Google Login

- Uses Google OAuth API
- User logs in using Google account
- No password required
- Google provides verified user details

7. Dashboard Page

After successful login:

- User is redirected to Dashboard
- Features:
    - File upload
    - View uploaded files

8. File Upload

- **Supported files:**
    - PDF
    - Images
    - Scanned documents
- Stored securely on server or cloud

Day-7

SQL (Structured Query Language)

What is SQL?

SQL is used to store, retrieve, update, and delete data in a database.

Database Systems:

- MySQL
- PostgreSQL
- SQLite

Used In:

- Websites
- Mobile Applications

Basic Database Operations (CRUD)

- Create → Create database/table
- Insert → Add data
- Read (Select) → Retrieve data
- Update → Modify data
- Delete → Remove data

important SQL Commands

- SELECT → Display records
- UPDATE → Modify records
- DELETE → Delete records
- DROP → Delete table/database
- RENAME → Rename table
- TRUNCATE → Remove all records from table

Comparison Operators

- = Equal
- != Not equal
- < Less than
- > Greater than

Logical Operators

- AND
- OR
- NOT

Sorting & Grouping

- ORDER BY → Sort results
- GROUP BY → Group similar records
- HAVING → Used with GROUP BY

Aggregate Functions

- MIN() → Minimum value
- MAX() → Maximum value
- AVG() → Average value
- COUNT() → Count records

Pattern Matching (LIKE)

- A% → Starts with A
- %a → Ends with a
- %a% → Contains a
- A%e → Starts with A and ends with e

Joins

- INNER JOIN → Common records
- LEFT JOIN → All left + matching right
- RIGHT JOIN → All right + matching left
- CROSS JOIN → All combinations

Other Operators

- IN → Match multiple values
- BETWEEN → Range
- LIKE → Pattern search

- IS NULL → Check null values
- UNION → Combine results (no duplicates)
- UNION ALL → Combine results (with duplicates)
- INTERSECT → Common records
- EXCEPT → Difference between queries

✅ **JWT (JSON Web Token)**

What is JWT?

JWT is a compact and secure way to send information between client and server as a signed token.

Used For:

- Authentication
- Authorization
- Stateless APIs

JWT Flow

1. User logs in (username/password)
2. Server validates credentials
3. Server generates JWT
4. Client stores token (localStorage / cookies)
5. Client sends token in request header
6. Server verifies token and gives access

OAuth

OAuth is an authentication system that allows login using:

- Google
- Facebook
- GitHub

Without sharing password with the application.

Day-8

Hashing

Hashing is a technique used to protect sensitive data like passwords.

What is Hashing?

- Hashing converts a password into a fixed-length, unreadable value
- It uses a mathematical algorithm
- This happens because of salting
- Salting adds extra security
- Hash cannot be converted back to the original password

Bcrypt Hashing (Python)

bcrypt is a Python library used for secure password hashing

Installation

```
pip install bcrypt
```

Import

```
import bcrypt
```

Hashing Example

```
import bcrypt
password = "rithwik".encode()
salt = bcrypt.gensalt()
hashed = bcrypt.hashpw(password, salt)
print(hashed)
```

Password Verification Example

```
entered_password = "rithwik".encode()
if bcrypt.checkpw(entered_password, hashed):
    print("Password matched")
else:
    print("Invalid password")
```

Python String Built-in Methods

strip()

Removes spaces from both sides

```
text = " rithwik "
print(text.strip())
```

join()

Joins elements using a separator

```
letters = ["g", "f", "k"]
print("".join(letters))
```

Output: gfk

replace()

Replaces part of a string

```
text = "I like python"
print(text.replace("python", "java"))
```

startswith()

Checks starting characters → True / False

endswith()

Checks ending characters → True / False

find()

Returns index of substring

isdigit()

Checks if string contains only numbers

```
"1235".isdigit()
```

isalpha()

Checks if string contains only letters

Dictionary Methods

Method	Use
get()	Get value using key
keys()	Returns all keys

Method	Use
values()	Returns all values
items()	Key-value pairs
pop("Age")	Removes specific key
popitem()	Removes last inserted item
clear()	Removes all items

Set Methods

Method	Use
add()	Adds element
remove()	Removes element (error if missing)
discard()	Removes safely
pop()	Removes random element
clear()	Empties set
union()	Combines sets
intersection()	Common elements
difference()	Elements in one set but not another

Day-8

SQL (Structured Query Language)

- Used to create, store, retrieve, and modify data in a database.

Basic SQL Commands

1. **CREATE –** Create a table
   CREATE TABLE table_name (column datatype);
2. **SELECT –** Retrieve data
   SELECT * FROM table_name;
3. **INSERT –** Add data
   INSERT INTO table_name VALUES (...);
4. **UPDATE –** Modify data
   UPDATE table_name SET column=value WHERE condition;
5. **DELETE –** Remove data
   DELETE FROM table_name WHERE condition;
6. **TRUNCATE –** Delete all records (faster)
   TRUNCATE TABLE table_name;
7. **DROP –** Delete entire table
   DROP TABLE table_name;

Operators & Patterns

- != or <> → Not equal
- A% → Starts with A
- %A → Ends with A
- %A% → Contains A

Functions

- **ORDER BY** → Sort data (ASC/DESC)

- **GROUP BY** → Group similar data
- **MIN()** → Smallest value
- **MAX()** → Largest value
- **AVG()** → Average value
- **COUNT()** → Number of rows

Joins

- Inner Join
- Left Join
- Right Join
- Cross Join
- Full Join

Other SQL Clauses

- **UNION** → Combine results (no duplicates)
- **UNION ALL** → Combine results (with duplicates)
- **IN** → Multiple values in condition
- **EXCEPT** → Values in first not in second
- **BETWEEN** → Range of values
- **LIKE** → Pattern matching
- **IS NULL** → Check empty values

Aggregate Functions

- **Work on multiple rows** → return single value
- **Examples:** SUM(), AVG(), COUNT(), MIN(), MAX()

```sql
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(100),
    emp_email VARCHAR(150)
);

CREATE TABLE departments (
    emp_id INT,
    dept_name VARCHAR(100)
);

INSERT INTO employees VALUES
(201, 'arjun', 'arjun.kumar@example.com'),
(202, 'meera', 'meera.sharma@example.com'),
(203, 'rahul', 'rahul.verma@example.com');
INSERT INTO employees VALUES
(204, 'arjun', 'arjun.work@example.com');

INSERT INTO departments VALUES
(201, 'Finance'),
(202, 'Marketing');

SELECT u.emp_id, u.emp_name, d.dept_name
FROM employees u
INNER JOIN departments d
ON u.emp_id = d.emp_id;

SELECT u.emp_id, u.emp_name, d.dept_name
FROM employees u
LEFT JOIN departments d
ON u.emp_id = d.emp_id;

SELECT emp_name, MIN(emp_id)
FROM employees
GROUP BY emp_name;
```

Output

Output:

```
emp_id      emp_name                                                                                                 dept_name
----------- -------------------------------------------------------------------------------------------------------- --------------------------------
       201 arjun                                                                                                    Finance
       202 meera                                                                                                    Marketing
emp_id      emp_name                                                                                                 dept_name
----------- -------------------------------------------------------------------------------------------------------- --------------------------------
       201 arjun                                                                                                    Finance
       202 meera                                                                                                    Marketing
       203 rahul                                                                                                    NULL
       204 arjun                                                                                                    NULL
emp_name
-------------------------------------------------------------------------------------------------------- -----------
arjun                                                                                                            201
meera                                                                                                            202
rahul                                                                                                            203
```

```sql
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(100),
    emp_email VARCHAR(150)
);

CREATE TABLE departments (
    emp_id INT,
    dept_name VARCHAR(100)
);

INSERT INTO employees VALUES
(201, 'arjun', 'arjun.kumar@example.com'),
(202, 'meera', 'meera.sharma@example.com'),
(203, 'rahul', 'rahul.verma@example.com');
INSERT INTO employees VALUES
(204, 'arjun', 'arjun.work@example.com');

INSERT INTO departments VALUES
(201, 'Finance'),
(202, 'Marketing');

SELECT u.emp_id, u.emp_name, d.dept_name
FROM employees u
INNER JOIN departments d
ON u.emp_id = d.emp_id;

SELECT u.emp_id, u.emp_name, d.dept_name
FROM employees u
LEFT JOIN departments d
ON u.emp_id = d.emp_id;

SELECT emp_name, MIN(emp_id)
FROM employees
GROUP BY emp_name;
SELECT
    MIN(emp_id),
    MAX(emp_id),
    AVG(emp_id),
    COUNT(emp_id)
FROM employees;
```

Output:

```
Output:
emp_id      emp_name                                                                                                 dept_name
----------- -------------------------------------------------------------------------------------------------------- --------------
       201 arjun                                                                                                    Finance
       202 meera                                                                                                    Marketing
emp_id      emp_name                                                                                                 dept_name
----------- -------------------------------------------------------------------------------------------------------- --------------
       201 arjun                                                                                                    Finance
       202 meera                                                                                                    Marketing
       203 rahul                                                                                                    NULL
       204 arjun                                                                                                    NULL
emp_name
-------------------------------------------------------------------------------------------------------------------- -----------
arjun                                                                                                                        201
meera                                                                                                                        202
rahul                                                                                                                        203

----------- ----------- ----------- -----------
       201         204         202           4
```

JWT (JSON Web Token)

- A secure and compact way to transfer data between client and server.
- Data is digitally signed, so it cannot be tampered with.

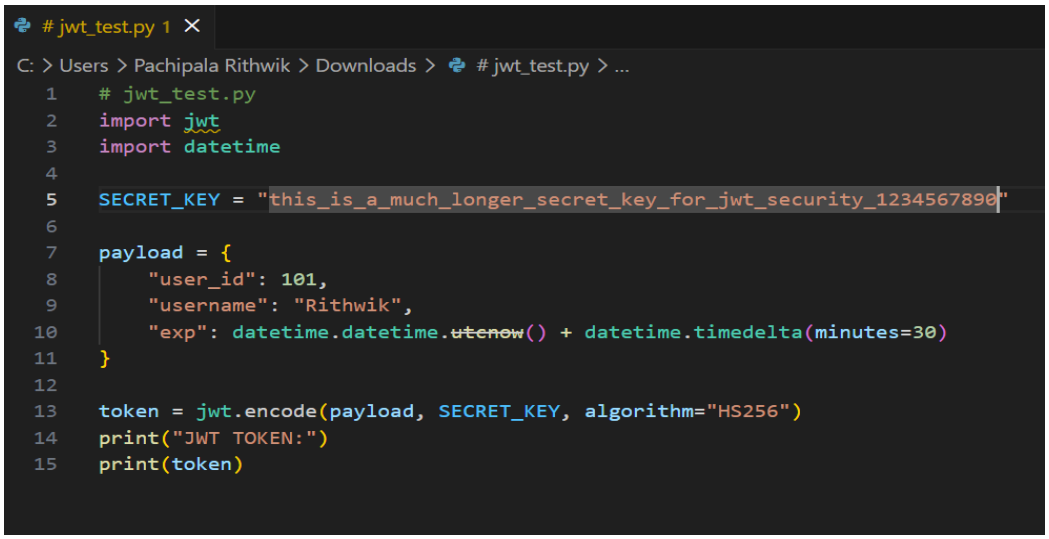
How JWT Works

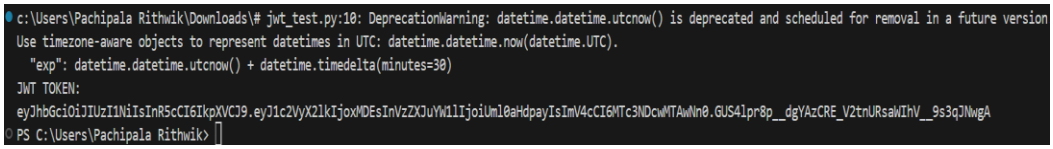
1. User logs in (username & password)
2. Server verifies credentials
3. Server creates a JWT
4. Client stores the token (cookies/local storage)
5. Client sends JWT with each request
6. Server verifies token and gives access

Implementation (Python)

- **Install:** pip install pyjwt
- **Import:** import jwt

Example:





Day-9

1. Hashing

- Converts password → fixed unreadable value
- One-way process (cannot reverse)
- Never store original passwords

2. Hashing vs Encoding vs Encryption

- Hashing → One-way, used for passwords (bcrypt, SHA-256)
- Encoding → Reversible, no security (Base64)
- Encryption → Two-way, uses key for security

3. bcrypt

- Secure password hashing algorithm
- Slow + uses salt → prevents attacks

4. Installation

- pip install bcrypt
- import bcrypt

5. Password & Bytes

- bcrypt uses bytes, not strings
- Example: password.encode()

6. Salt

- Random data added before hashing
- Ensures same passwords → different hashes
- Prevents hacking attacks

7. Hashing Example

- bcrypt.hashpw(password_bytes, bcrypt.gensalt())

- Store hashed value in DB

8. Summary

- Never store plain passwords
- Use hashing (bcrypt best)
- Hashing → irreversible
- Salt → adds security

String Methods

- strip() → remove spaces
- join() → combine strings
- replace() → replace text
- startswith() / endswith() → check start/end
- find() → get index
- isdigit() → numbers only
- isalpha() → letters only

Dictionary Methods

- get() → safe access
- keys() → all keys
- values() → all values
- items() → key-value pairs
- update() → add/modify
- pop() → remove by key
- popitem() → remove last
- clear() → remove all

Set Methods- add( ), remove( ), discard( ),pop( ), union( ) etc.

Day-10

File input and output methods

- **open():** Opens a file
- **read():** Reads the entire file or a specified number of bytes from the file
- **readline():** Reads one line from the file at a time**.**
- **readlines():** Reads all lines into a list of strings.
- **write():** Writes a string or bytes to the file (requires the file to be opened in write, append, or exclusive creation mode).
- **Writelines():** writes a sequence of strings, though it requires manually adding newline characters to each string in the sequence.

close(): Closes the opened file.

Common Built-in Functions

- **len()** → Number of items
- **range()** → Sequence of numbers
- **print()** → Display output
- **type()** → Data type
- **id()** → Unique object ID
- **uuid()** → Generate unique ID
- **sorted()** → Returns sorted list
- **enumerate()** → Adds index to items
- **zip()** → Combine iterables
- **join()** → Join elements into string

Conversion functions:

- **int():** Converts a number or string to an integer (int(10.5) -> 10).

- **float():** Converts an integer or string to a floating-point number (float("10.5") -> 10.5).
- **str():** Converts any data type to its string representation (str(10) -> "10").
- **list():** Converts an iterable (string, tuple, set) into a list (list("abc") -> ['a', 'b', 'c']).
- **dict():** Converts a sequence of key-value pairs (like a list of tuples) into a dictionary (dict([('a', 1)]) -> {'a': 1}).
- **tuple():** Converts an iterable (list, string, set) into a tuple (tuple([1, 2]) -> (1, 2)).
- **set():** Converts an iterable into a set, removing duplicates (set([1, 2, 2]) -> {1, 2}).

Mathematical Functions

- **max()** → Largest value
- **pow(x, y)** → x raised to power y
- **round()** → Round number
- **filter()** → Select items based on condition

```python
# 1. len()
language = "JavaScript"
print("Length of string:", len(language))

# 2. range()
print("\nUsing range():")
for i in range(3, 9):
    print(i, end=" ")

# 3. id()
y = 250
print("\n\nMemory address of y:", id(y))

# 4. power() → using pow()
base = 3
exp = 4
print("\nPower (3^4):", pow(base, exp))
```

```
Length of string: 10

Using range():
3 4 5 6 7 8

Memory address of y: 140307920503272

Power (3^4): 81

=== Code Execution Successful ===
```

```python
# 5. sorted()
numbers = [12, 4, 8, 15, 3]
print("Sorted list:", sorted(numbers))

# 6. join()
words = ["Learning", "Python", "is", "fun"]
sentence = "-".join(words)
print("Joined string:", sentence)

# 7. Convert string into list
text = "World"
char_list = list(text)
print("String to list:", char_list)

# 8. round()
value = 9.87654
print("Rounded value:", round(value, 3))
```

```
Sorted list: [3, 4, 8, 12, 15]
Joined string: Learning-Python-is-fun
String to list: ['W', 'o', 'r', 'l', 'd']
Rounded value: 9.877

=== Code Execution Successful ===
```

Day 11 and day 12:

Milestone-1

It includes the Group presentation and the Group's project discussion.

It includes individual Members task.

Day13:

Class methods

- **getattr(obj, name[, default**]): Returns the value of a named attribute from an object, optionally returning a default value if not found.
- **setattr(obj, name, value):** Assigns a new value to a named attribute of an object, creating the attribute if it does not exist**.**
- **hasattr(obj, name):** Returns True if the object has the specified attribute, otherwise returns False.
- **delattr(obj, name):** Deletes a named attribute from an object, equivalent to the del statement.
- **isinstance(obj, classinfo):** Checks if an object is an instance of a specified class or a tuple of classes, returning True or False.
- **Issubclass:** checks if a class is a subclass of another class (or a tuple of classes).

Miscellaneous functions

- **globals()** → Returns a dictionary of the current global symbol table.
- **locals()** → Returns a dictionary of the current local symbol table.
- **callable(obj)** → Returns True if the object can be called like a function, else False.
- **eval(expr)** → Evaluates a Python expression (string) and returns its result.
- **exec(code)** → Executes Python statements dynamically from a string or code object.

Exception Handling

- **try** → Code that may cause error
- **except** → Handles the error
- **finally** → Always executes

Assignment – 3

enumerate() Function

Use:

When you need both **index (position)** and **value** while looping.

Works with:

Lists, strings, tuples

Without vs With enumerate()

Without enumerate():

- Uses range(len(x))
- More code
- Error-prone

With enumerate():

- Uses enumerate(x)
- Cleaner & shorter
- Safer

```python
print("from start=0")
names = ['rithwik', 'vamsi', 'ajay']
for indx, name in enumerate(names, start=0):
    print(f"{indx} . {name}")

print("from start=1")
for indx, name in enumerate(names, start=1):
    print(f"{indx} . {name}")

# default start=0
string = "coding"
for ind, ch in enumerate(string):
    print(f"{ind} . {ch}")

# list of enumerate
names = ['rithwik', 'vamsi', 'ajay']
new_names = list(enumerate(names))
print(new_names)

# directly print without using variable
print(list(enumerate((100, 200, 300))))
```

```
from start=0
0 . rithwik
1 . vamsi
2 . ajay
from start=1
1 . rithwik
2 . vamsi
3 . ajay
0 . c
1 . o
2 . d
3 . i
4 . n
5 . g
[(0, 'rithwik'), (1, 'vamsi'), (2, 'ajay')]
[(0, 100), (1, 200), (2, 300)]

=== Code Execution Successful ===
```

isinstance()

- Checks if an object belongs to a class/data type
- **Syntax:** isinstance(object, classinfo)
- Returns **True / False**

Example:

```
x = 10
print(isinstance(x, int))   # True

class Student:
    pass

s = Student()
print(isinstance(s, Student))   # True
```

GC Library (Garbage Collection)

- **gc = Garbage Collection**
- Automatically removes unused objects
- Frees memory

```python
import gc

# Check if garbage collector is active
print("Garbage Collector Enabled:", gc.isenabled())

# Example 1: Reference counting handles cleanup automatically
list1 = [10, 20, 30]
list2 = [40, 50, 60]

# Remove references
list1 = None
list2 = None

# Trigger garbage collection
collected_items = gc.collect()
print("Objects collected (no circular refs):", collected_items)

# Example 2: Circular references require gc
class Alpha:
    pass

class Beta:
    pass

obj1 = Alpha()
obj2 = Beta()

# Create circular reference
obj1.link = obj2
obj2.link = obj1

# Delete references
del obj1
del obj2

# Force garbage collection
print("Objects collected (with circular refs):", gc.collect())
```

```
Garbage Collector Enabled: True
Objects collected (no circular refs): 0
Objects collected (with circular refs): 2

=== Code Execution Successful ===
```

Day14:

AI Models

- AI models are **trained programs** designed for specific tasks
- Common uses:
    - **Chat** → answering questions
    - **Prediction** → e.g., weather forecasting
    - **Image generation / vision** → tools like OpenCV

Major AI Platforms

- **Google** → Gemini
- **OpenAI** → GPT
- **Microsoft** → Copilot (GPT-based)
- **Anthropic** → Claude

- **Meta** → LLaMA
- **Amazon** → Bedrock (multi-model platform)

Large Language Models (LLMs)

- LLMs are **AI systems trained on huge text data**
- Capabilities:
    - Understand & generate text
    - Translate languages
    - Summarize content
    - Answer questions
- They work by **predicting the next word/token**

Key Concepts

- **Token Limit** → Maximum input + output size per request
- **Rate Limit** → How many requests you can make in a time period

Chat Completion (Example Idea)

- Used to generate responses from AI
- Key parameters:
    - **model** → which AI model to use
    - **messages** → input conversation
    - **temperature (0–1+)** → controls creativity
    - **max_tokens** → response length

Higher temperature = **more creative output**
Lower temperature = **more accurate output**

Creating APIs (Simple Idea)

To use AI models, you need APIs:

- OpenAI API
- Gemini API
- LLaMA API

DAY-15

NLP (Natural Language Processing)

- NLP is a subfield of AI that helps computers read, understand, and process human language
- **Used in:**
    - Chatbots
    - Translation
    - Sentiment analysis

Types of sentences:

- Positive sentence
- Negative sentence

NLTK (Natural Language Toolkit)

- A Python library for working with text and language data
- Used for text processing & NLP tasks

7 Key Features of NLTK:

Tokenization

- Splitting text into words/sentences (tokens)

```
from nltk.tokenize import word_tokenize
```

Stop Word Removal

- Removes common words (is, are, in, and)
- Helps focus on important words

from nltk.corpus import stopwords

Stemming

- Reduces words to root form
- Root may not be meaningful

Example: *playing → play*

from nltk.stem import PorterStemmer

Lemmatization

- Converts word to dictionary (meaningful) form

Example: *better → good*

from nltk.stem import WordNetLemmatizer

POS Tagging (Parts of Speech)

- Identifies grammar role:
  - Noun, Verb, Adjective, etc.

from nltk.tokenize import word_tokenize

NER (Named Entity Recognition)

- Identifies:
  - Names
  - Places
  - Organizations

nltk.download('maxent_ne_chunker')

Text Preprocessing

Steps involved:

- Lowercasing text
- Removing punctuation
- Removing stop words
- Tokenization
- Stemming/Lemmatization

Assignment-4

```python
import nltk
import string   # It contains all punctuation symbols: instead of manually typing symbols.
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk import pos_tag, ne_chunk

# -------- DOWNLOAD ONLY REQUIRED RESOURCES --------
nltk.download('punkt')
nltk.download('punkt_tab')   # REQUIRED for Python 3.12
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
nltk.download('maxent_ne_chunker')
nltk.download('words')

print("PROGRAM STARTED\n")

# -------- SAMPLE TEXT --------
text = "Hi my name is Rithwik and I'm currently pursuing my B-Tech 2nd year in VRSEC."

# -------- TEXT PREPROCESSING --------

# 1. Tokenization
tokens = word_tokenize(text)

# 2. Lowercasing
tokens = [word.lower() for word in tokens]
```

```python
# 3. Remove punctuation
tokens = [word for word in tokens if word not in string.punctuation]

# 4. Remove stopwords (i and my are considered as stop words so not appear in output)
stop_words = set(stopwords.words('english'))

with_stopwords = tokens
only_stopwords = [w for w in tokens if w in stop_words]
without_stopwords = [w for w in tokens if w not in stop_words]

# 5. Stemming
stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(word) for word in tokens]

# 6. Lemmatization
lemmatizer = WordNetLemmatizer()
lemmatized_words = [lemmatizer.lemmatize(word) for word in tokens]

# lemmatizer.lemmatize("pursuing", pos="v") o/p => pursue

print("Tokens:", tokens)
print("Only stopwords:", only_stopwords)
print("Without stopwords:", without_stopwords)
print("Stemmed:", stemmed_words)
print("Lemmatized:", lemmatized_words)

# -------- NER (NAMED ENTITY RECOGNITION) --------
pos_tags = pos_tag(word_tokenize(text))
named_entities = ne_chunk(pos_tags)

print("\nNamed Entities:")
print(named_entities)

print("\nPROGRAM ENDED")
```

Output:

```
Tokens: ['hi', 'my', 'name', 'is', 'rithwik', 'and', 'i', "'m", 'currently', 'pursuing', 'my', 'b-tech', '2nd', 'year', 'in', 'vrsec']
Only stopwords: ['my', 'is', 'and', 'i', 'my', 'in']
Without stopwords: ['hi', 'name', 'rithwik', "'m", 'currently', 'pursuing', 'b-tech', '2nd', 'year', 'vrsec']
Stemmed: ['hi', 'my', 'name', 'is', 'rithwik', 'and', 'i', "'m", 'current', 'pursu', 'my', 'b-tech', '2nd', 'year', 'in', 'vrsec']
Lemmatized: ['hi', 'my', 'name', 'is', 'rithwik', 'and', 'i', "'m", 'currently', 'pursuing', 'my', 'b-tech', '2nd', 'year', 'in', 'vrsec']
```

Machine Learning (ML)

Machine Learning is a branch of AI where systems learn from data automatically and make decisions or predictions without hard-coded rules.

📊 Types of Machine Learning

1.Supervised Learning

- Trained using input + correct output (labeled data)
- Learns mapping between them
  Example: email spam filter, exam score prediction

2. Unsupervised Learning

- Works with data without labels
- Finds hidden patterns or groups
  Example: customer segmentation, data grouping

3. Reinforcement Learning

- Learns through trial and error
- Uses rewards (good) & penalties (bad)
  Example: game AI, robots, autonomous driving

Embeddings

- Data (text/image) is converted into numeric vectors
- Helps machines understand meaning & similarity
  Example: similar words have similar vectors

Model Problems

Underfitting

- Model is too simple
- Fails to capture patterns

- Poor performance on both training & testing

Overfitting

- Model memorizes instead of learning
- Works well on training data but fails on new data

Logistic Regression (Classification)

- Used for binary decisions (Yes/No, 0/1)
- Steps:
    1. Takes input features
    2. Computes a score
    3. Converts it into probability (0–1)
    4. Applies threshold (usually 0.5)

Linear Regression (Prediction)

- Used to predict continuous values (numbers)
- Fits a straight line through data

Formula:

$$y = mx + c$$

- Example: predicting salary, marks, price

scikit-learn vs sklearn

- scikit-learn → actual library name
- sklearn → used in Python code

Correct usage:

pip install scikit-learn

import sklearn

Decision Trees

- Works like a flowchart of questions
- Each step:
    - Ask a condition
    - Split data into branches
    - Continue until final decision

OCR (Optical Character Recognition)

Used to convert images/text in photos into machine-readable text

Popular OCR Tools:

- PaddleOCR
- PyTesseract

  DocTR
- Surya OCR
- EasyOCR

Day-16:

Steps:

```python
# Simulated OCR Program (works in Programiz)

print("PROGRAM STARTED\n")

# Sample "image text" (pretend this came from an image)
image_text = "Invoice No: 12345\nTotal Amount: 5000\nDate: 12-03-2026"

# Step 1: Tokenization (split into words)
tokens = image_text.split()

# Step 2: Lowercase
tokens = [word.lower() for word in tokens]

# Step 3: Remove simple punctuation
clean_tokens = []
for word in tokens:
    word = word.replace(":", "").replace("\n", "")
    clean_tokens.append(word)

# Step 4: Display results
print("Extracted Text:")
print(image_text)

print("\nTokens:")
print(clean_tokens)

print("\nPROGRAM ENDED")
```

Output:

```
PROGRAM STARTED

Extracted Text:
Invoice No: 12345
Total Amount: 5000
Date: 12-03-2026

Tokens:
['invoice', 'no', '12345', 'total', 'amount', '5000', 'date', '12-03-2026']

PROGRAM ENDED

=== Code Execution Successful ===
```

Day-17:

K-Means

- Unsupervised clustering algorithm
- Groups data into K clusters
- Each cluster has a centroid (center)
- Steps: choose K → assign points → update centroid → repeat
- Goal: minimize WCSS (tight clusters)

Euclidean Distance

- Measures distance between points
- Formula: $\sqrt{((x_2-x_1)^2 + (y_2-y_1)^2)}$
- Smaller distance = more similarity

WCSS (Within-Cluster Sum of Squares)

- Measures cluster compactness
- Small WCSS = better clusters
- Used to evaluate K-Means

Elbow Method

- Finds best K value
- Plot: K vs WCSS
- Choose point where graph bends (elbow)

KNN (K-Nearest Neighbours)

- Supervised learning algorithm
- Uses nearest K points to predict

Classification:

- Output = category

- Uses majority vote

Regression:

- Output = number
- Uses average value

🔹 Backpropagation

- Used to train neural networks
- Reduces prediction error

Steps:

1. Forward pass → predict output
2. Calculate loss (error)
3. Backward pass → compute gradients
4. Update weights using:
   New weight = Old weight − Learning rate × Gradient